Array

leetcode

53.最大子数组和

- 一个整数数组 `nums` ，找出具有最大和的连续子数组。

```
输入: nums = [-2,1,-3,4,-1,2,1,-5,4]
输出: 6
解释: 连续子数组 [4,-1,2,1] 的和最大，为 6 。
```

- 思路：动态规划问题。$dp[i]$代表以第i个数结尾的连续子数组的最大和。我们需要找出$max(dp[i])$。状态转移方程为$dp[i] = max\{dp[i-1] + nums[i], nums[i]\}$.

```cpp
class Solution {
public:
    int maxSubArray(vector<int>& nums) {
        int n = nums.size();
        vector<int> dp(n);
        // 初始条件
        dp[0] = nums[0];
        int maxVal = nums[0];
        for( int i=1; i<n; i++){
            dp[i] = std::max(dp[i-1]+nums[i],nums[i]); // 状态转移方程
            maxVal = std::max(maxVal,dp[i]); // 存储dp中的最大值
        }
        return maxVal;
    }
};
```

-

56.合并区间

- $intervals$ 表示若干个区间的集合，其中单个区间为 $intervals[i] = [start_i, end_i]$，合并所有重叠的区间，并返回**一个不重叠的区间数组**。
- 思路：先将区间数组按左端点进行升序排序。然后进行区间合并。
  - 如果当前区间的左端点在数组 `merged` 中最后一个区间的右端点之后，那么它们不会重合。
  - 否则，两区间重合。需要用当前区间的右端点更新数组 `merged` 中最后一个区间的右端点，将其置为二者的较大值。

```
[(1, 9), (2, 5), (19, 20), (10, 11), (12, 20), (0, 3), (0, 1), (0, 2)]
```

sort

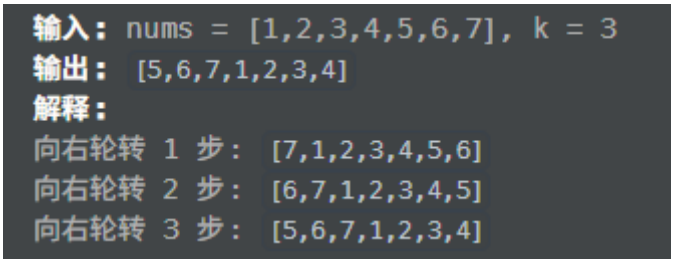
```
[(0, 3), (0, 1), (0, 2), (1, 9), (2, 5), (10, 11), (12, 20), (19, 20)]
```

```
class Solution {
public:
    vector<vector<int>> merge(vector<vector<int>>& intervals) {
        // 对区间按照左端点进行升序排列
        sort(intervals.begin(),intervals.end());
        vector<vector<int>> merged;
        merged.push_back(intervals[0]); // 将第一个区间加入 merged 数组中
        for( int i=1; i<intervals.size(); i++){
            int lVal = intervals[i][0],rVal = intervals[i][1];
            int mergeRightVal = merged.back()[1];
            if(mergeRightVal < lVal){ // 当前新区间的左端点大于merged后的区间的右端点，不可以合并
                merged.push_back({lVal,rVal});
            }else{ // 可以合并，比较新区间的右端点和merged区间的右端点，选择最大的那个
                merged.back()[1] = max(merged.back()[1],rVal);
            }
        }
        return merged;
    }
};
```

-

189.轮转数组

- 整数数组 `nums`，将数组中的元素向右轮转 `k` 个位置，其中 `k` 是非负数。



- 思路：三次反转法。
  - **整体反转**：将整个数组颠倒过来。此时，原本在末尾的 `k` 个元素跑到了数组的最前面，但它们的内部顺序是反的；原本在前面的元素跑到了后面，内部顺序也是反的。
  - **局部反转（前 k 个）**：将现在位于最前面的 `k` 个元素再次反转，恢复它们原本的顺序。
  - **局部反转（剩余部分）**：将剩下的 `n-k` 个元素再次反转，恢复它们原本的顺序。

```
class Solution {
public:
    void reverse(vector<int>& nums,int start,int end){
        while( start < end ){
            swap(nums[start],nums[end]);
            start += 1;
            end -= 1;
        }
    }

    void rotate(vector<int>& nums, int k) {
        k %= nums.size(); // 实际移动次数
        reverse(nums,0,nums.size()-1); // 先整体交换
        reverse(nums,0,k-1); // 在交换子序列
```

```
        reverse(nums,k,nums.size()-1);
    }
};
```

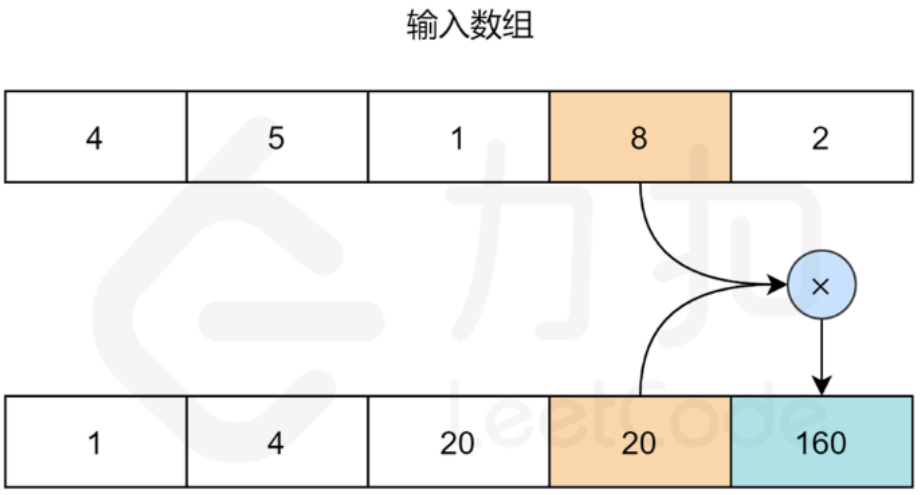
-

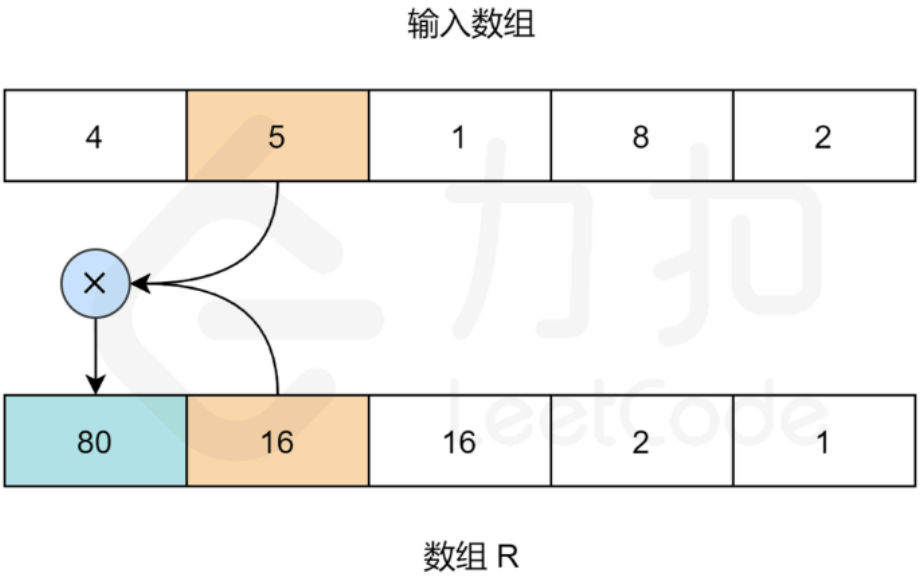
238.除了自身意外数组的乘积

- 一个整数数组 `nums`，返回 数组 `answer` ，其中 `answer[i]` 等于 `nums` 中除了 `nums[i]` 之外其余各元素的乘积 。

```
输入： nums = [1,2,3,4]
输出： [24,12,8,6]
```

- 思路：answer[i]处的值可以利用索引**左侧所有数字的乘积**和**右侧所有数字的乘积**（即前缀与后缀）相乘得到答案。遍历两次，第一次构建左侧乘积L，第二次构建右侧乘积R。然后根据R、L构建answer。






```
class Solution {
public:
    vector<int> productExceptSelf(vector<int>& nums) {
        int length = nums.size();
        vector<int> L(length);
        vector<int> R(length);
```

```
        // 构建L[i]，索引i左侧所有元素的乘积
        L[0] = 1;
        for( int i=1;i<length;i++){
            L[i] = L[i-1] * nums[i-1];
        }
        // 构建R[i]，索引i右侧所有元素的乘积
        R[length-1] = 1;
        for( int i=length-2;i>=0;i--){
            R[i] = R[i+1] * nums[i+1];
        }
        // 根据L和R，构建answer，排除自身的其余所有元素的乘积
        vector<int> answer(length);
        for( int i=0; i<length;i++){
            answer[i] = L[i] * R[i];
        }
        return answer;
    }
};
```

-